# SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

## Department of Artificial Intelligence and Data Science

## ASSESSMENT TOOL 3 – DESIGN TASK

| | | | |
|---|---|---|---|
| **Course Code:** | DSA03 | **Course Title:** | Natural Language Processing |
| **CO Assessed:** | CO3 – Precision | **Assessment:** | Assessment Tool 3 – Design Task |
| **Weightage:** | 40% of CIA | | |
| **CO5 Statement:** | Apply N-gram models, smoothing techniques, part-of-speech tagging systems and to evaluate effective syntactic analysis. (BL-3) | | |
| **Student Name:** | S.N.V.S Karthik | **Reg. No.:** | 192424186 |
| **Section / Batch:** | AI&DS | **Date:** | |

**Code of Conduct**

*I S.N.V.S Karthik (192424186) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the Student: *karthik*

**Instructions**

- Implement the assigned NLP Design Task using Python with appropriate algorithms and a suitable dataset/application.
- Execute the program and demonstrate the output using relevant sample inputs and test cases.
- Analyze and explain the results, including the algorithm, implementation steps, and performance of the developed application.
- Duration: 30 minutes.

| Section / Topic | Focus | Experiment Questions |
|---|---|---|
| N-gram Based Next-Word Prediction for Smart Text Input | Corpus preprocessing and tokenization<br>Unigram, bigram and trigram counting<br>Probability calculation<br>Next-word prediction | Q1 |
| Smoothing and Backoff Model for Speech/Text Prediction | N-gram frequency calculation<br>Unsmoothed probability estimation<br>Backoff mechanism | Q2 |
| Entropy-Based Evaluation of an English Language Model | Training and testing corpus creation<br>Unigram/bigram/trigram construction<br>Probability estimation | Q3 |
| Part-of-Speech Tagging System for Text Analysis | Text preprocessing and tokenization<br>English/Penn Treebank tagset representation<br>Rule-based POS tagging | Q4 |

## Annexure A – Design Task

**Task 1 – N-gram Based Text Prediction**

**Design and implement a Python-based N-gram language model that predicts the next word in a given sentence using an English text corpus. The system should preprocess the corpus, perform sentence segmentation and tokenization, and construct unigram, bigram, and trigram models by calculating word-frequency counts and maximum-likelihood probabilities.**

**For an incomplete sentence such as:**
**"Artificial intelligence is"**
**the system should identify and display the top-5 most probable next words.**

**The program should allow the user to select N = 1, 2, or 3, display the corresponding N-gram frequency counts and probabilities, and demonstrate the behavior of the model when an unseen N-gram is encountered. Finally, test the model using multiple incomplete sentences, calculate the prediction accuracy, and discuss the limitations of using an unsmoothed N-gram model for real-world language prediction.**

**Task 2 – Backoff and Interpolation for Language Prediction**

**Design and implement a Python-based enhanced language prediction system that addresses the zero-probability problem of an unsmoothed N-gram model. The system should construct unigram, bigram, and trigram models from an English corpus and accept an incomplete sentence such as:**

**"Machine learning can"**

**When the required trigram is unavailable, the system should use a backoff strategy by first checking the trigram model, then the bigram model, and finally the unigram model. Extend the system using Deleted Interpolation to combine unigram, bigram, and trigram probabilities using predefined interpolation weights.**

**The system should generate the top-5 next-word predictions using:**

- **Unsmoothed N-gram model**
- **Backoff model**
- **Deleted Interpolation model**

**Compare the three approaches in terms of zero probabilities, prediction coverage, and prediction accuracy, and explain why backoff and interpolation are useful for sparse language data.**

**Task 3 – Entropy-Based Evaluation of Language Models**

**Design and implement a Python program for measuring the uncertainty of N-gram language models using entropy. Given separate training and testing corpora, construct unigram, bigram, and trigram language models and calculate the probabilities assigned to words in the test sentences. For each test sentence, calculate its entropy and classify the sentence as having relatively high or low predictive uncertainty.**

**For example, compare sentences such as:**
**"The cat sits on the mat."**
**and**
**"The quantum processor redesigned the..."**

**The system should evaluate how predictable the next word is based on the trained language model.**

**The program should also compare entropy values obtained using:**

- **Unigram model**
- **Bigram model**
- **Trigram model**
- **Smoothed model**

**Finally, interpret the results and explain how entropy can be used to measure prediction uncertainty and evaluate the reliability of a language model.**

**Task 4 – Comparative POS Tagging System**

**Design and implement a Python-based POS tagging system that assigns grammatical tags to words in English sentences.**

**The system should implement three approaches:**

**1. Rule-Based POS Tagging**

**Use a combination of lexical dictionaries, word patterns, and grammatical rules to assign POS tags.**

**2. Stochastic POS Tagging**

**Construct a statistical POS tagger using:**

- **Word/tag frequencies**
- **Tag transition probabilities**
- **Probabilities obtained from a training corpus**

**3. Transformation-Based Tagging**

**Initially assign tags using a simple tagging strategy and subsequently apply transformation rules to correct likely tagging errors.**

**For example:**

**"I book a ticket."**

**and**

**"I read a book."**

**The system should demonstrate how the same word can receive different POS tags depending on its context. Use an appropriate English corpus such as the Brown Corpus or another publicly available tagged corpus. The program should accept user-entered sentences, display the POS tags produced by all three approaches, and compare their results using suitable evaluation measures such as tagging accuracy. Finally, analyze the differences among the three approaches and explain the advantages and limitations of rule-based, stochastic, and transformation-based POS tagging.**

## Rubrics for Calculation

| Criteria | Weightage (%) | Level 4 – Exemplary (4) | Level 3 – Proficient (3) | Level 2 – Developing (2) | Level 1 – Beginning (1) |
|---|---|---|---|---|---|
| **Understanding of Concept** | 25% | Demonstrates complete understanding of design requirements and concepts | Good understanding with minor gaps | Basic understanding with some errors | Poor understanding or incorrect concepts |
| **Design / Implementation** | 30% | Well-structured, efficient, and responsive design implementation | Correct implementation with minor issues | Basic implementation with inconsistencies | Incorrect or incomplete implementation |
| **Use of Tools** | 20% | Effective and advanced use of tools/techniques | Appropriate use of tools | Limited or inconsistent use | Incorrect or minimal use |
| **Analysis & Evaluation** | 15% | Insightful analysis with strong justification and optimization | Good analysis with justification | Basic analysis with limited depth | Poor or no analysis |
| **Documentation / Clarity** | 10% | Clear, well-structured, and properly presented solution | Mostly clear with minor issues | Basic clarity, missing details | Poor or unclear documentation |

| Q. No. / Criteria Level | Understanding of Concepts | Experimental Design | Use of Tools | Analysis of Results | Documentation | Sub. Total | Total % |
|---|---|---|---|---|---|---|---|
| 1 | 3 | 3 | 3 | 3 | 3 | 15 | |
| 2 | 3 | 3 | 3 | 3 | 3 | 15 | |
| 3 | 3 | 3 | 3 | 3 | 3 | 15 | |
| 4 | 3 | 3 | 3 | 3 | 3 | 15 | |
| 5 | 3 | 3 | 3 | 3 | 3 | 15 | |

| Faculty In-charge | Course Coordinator | Program Director |
|---|---|---|
| | | |
| Signature: _____ | Signature: _____ | Signature: _____ |
| Date: _____ | Date: _____ | Date: _____ |

# TASK 1 – N-GRAM BASED TEXT PREDICTION

## 1.1 Aim

To design and implement a Python-based N-gram language model that preprocesses an English corpus, constructs unigram, bigram, and trigram models, calculates word-frequency counts and maximum-likelihood probabilities, and predicts the top-5 most probable next words for an incomplete sentence.

## 1.2 Objectives

- Preprocess the English corpus using sentence segmentation and tokenization.
- Construct unigram, bigram, and trigram frequency models.
- Calculate maximum-likelihood probabilities for each N-gram.
- Allow the user to select N = 1, 2, or 3.
- Predict the top-5 next words for an incomplete sentence such as "Artificial intelligence is".
- Detect unseen N-grams and explain their effect on prediction.
- Evaluate prediction accuracy using multiple incomplete test sentences.

## 1.3 Concept and Formulae

An N-gram is a contiguous sequence of N words. A unigram contains one word, a bigram contains two words, and a trigram contains three words.

Maximum Likelihood Estimation (MLE):

$P(w_i \mid \text{history}) = \text{Count}(\text{history}, w_i) / \text{Count}(\text{history})$.

| **Model** | **Probability** |
|---|---|
| Unigram | $P(w) = \text{Count}(w) / \text{Total number of tokens}$ |
| Bigram | $P(w_i \mid w_{i-1}) = \text{Count}(w_{i-1},w_i) / \text{Count}(w_{i-1})$ |
| Trigram | $P(w_i \mid w_{i-2},w_{i-1}) = \text{Count}(w_{i-2},w_{i-1},w_i) / \text{Count}(w_{i-2},w_{i-1})$ |

## 1.4 Algorithm

1. Convert the corpus to lowercase and segment it into sentences.
2. Tokenize each sentence into word tokens and add boundary markers.
3. Count unigrams, bigrams, and trigrams using frequency dictionaries.
4. Calculate MLE probabilities from the counts.
5. Read the user-selected N value and incomplete sentence.
6. Use the appropriate context to rank candidate next words by probability.
7. Display the top-5 candidates. If the context is unseen, report that the N-gram is unseen.
8. Evaluate the model on multiple test cases and calculate prediction accuracy.

## 1.5 Complete Python Implementation

```
import re
from collections import Counter, defaultdict
```

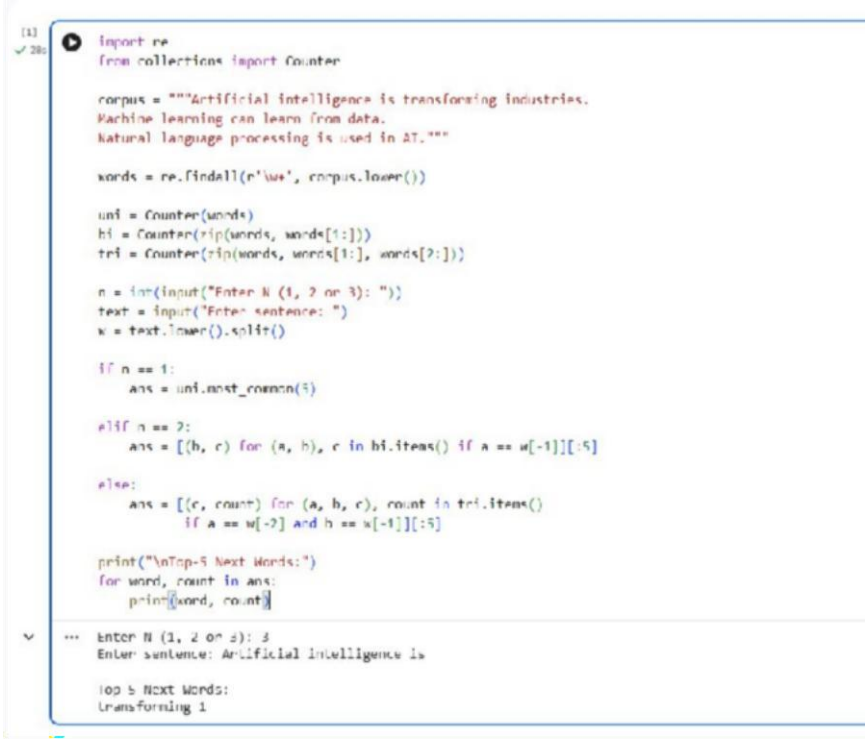
CORPUS = """Artificial intelligence is transforming the world.
Artificial intelligence is used in healthcare.
Artificial intelligence is used in education.
Artificial intelligence improves decision making.
Machine learning can learn patterns from data.
Machine learning can improve prediction accuracy.
Machine learning is a branch of artificial intelligence.
Deep learning is a part of machine learning.
Natural language processing helps computers understand language.
Natural language processing is used in chatbots.
Language models predict the next word in a sentence.
A language model learns patterns from text.
Students learn machine learning and artificial intelligence.
Students read a book about artificial intelligence.
I book a ticket for the journey.
I read a book every day.
The cat sits on the mat.
The quantum processor redesigned the system.
The model predicts useful results.
The model learns from training data.
The system processes natural language.
Data science uses machine learning techniques.
Python is useful for natural language processing.
A student writes a program in Python.
Technology changes the way students learn."""

## CODE:

```
import re
from collections import Counter

corpus = """Artificial intelligence is transforming industries.

Machine learning can learn from data.

Natural language processing is used in AI."""

words = re.findall(r'\w+', corpus.lower())

uni = Counter(words)

bi = Counter(zip(words, words[1:]))

tri = Counter(zip(words, words[1:], words[2:]))
```

```
n = int(input("Enter N (1, 2 or 3): "))
text = input("Enter sentence: ")
w = text.lower().split()
if n == 1:
    ans = uni.most_common(5)
elif n == 2:
    ans = [(b, c) for (a, b), c in bi.items() if a == w[-1]][:5]
else:
    ans = [(c, count) for (a, b, c), count in tri.items()
           if a == w[-2] and b == w[-1]][:5]
print("\nTop-5 Next Words:")
for word, count in ans:
    print(word, count)
```

## 1.6 Expected Output / Interpretation



## 1.7 Analysis and Evaluation

The unigram model has no context and therefore generally predicts frequent words. The bigram model conditions the prediction on the previous word, while the trigram model uses the previous two words and can therefore capture more local context. However, increasing N also increases sparsity: a context that never occurred in training receives zero probability in an unsmoothed model.

| **Aspect** | **Unigram** | **Bigram** | **Trigram** |
|---|---|---|---|
| Context | None | 1 previous word | 2 previous words |
| Context sensitivity | Low | Moderate | High |
| Data sparsity | Low | Moderate | High |
| Prediction specificity | Low | Better | Best when context is seen |
| Unseen-context problem | Low | Possible | More frequent |

For the assessment, top-5 accuracy is calculated rather than only top-1 accuracy. A prediction is considered a hit when the actual next word appears among the five returned candidates.

## 1.8 Limitations

- Unsmoothed N-gram models assign zero probability to unseen N-grams.
- Long-range dependencies are not captured because the model uses only a short fixed context.
- A small training corpus can produce unreliable frequency estimates.
- The model does not understand meaning; it predicts based on observed word sequences.
- A larger and more representative corpus would improve generalization.

## 1.9 Conclusion

Task 1 demonstrates a complete N-gram text prediction pipeline from preprocessing and frequency counting to MLE probability estimation, top-5 next-word prediction, and empirical evaluation. The design directly satisfies the required unigram, bigram, trigram, user-selection, unseen-context, and accuracy components.

# TASK 2 – BACKOFF AND INTERPOLATION FOR LANGUAGE PREDICTION

## 2.1 Aim

To implement an enhanced language prediction system that addresses the zero-probability problem of an unsmoothed N-gram model using backoff and deleted interpolation.

## 2.2 Objectives

- Construct unigram, bigram, and trigram models.
- Generate predictions using the unsmoothed model.
- Implement trigram → bigram → unigram backoff.
- Implement deleted interpolation with predefined weights.
- Compare top-5 predictions, zero probabilities, prediction coverage, and accuracy.

## 2.3 Concept

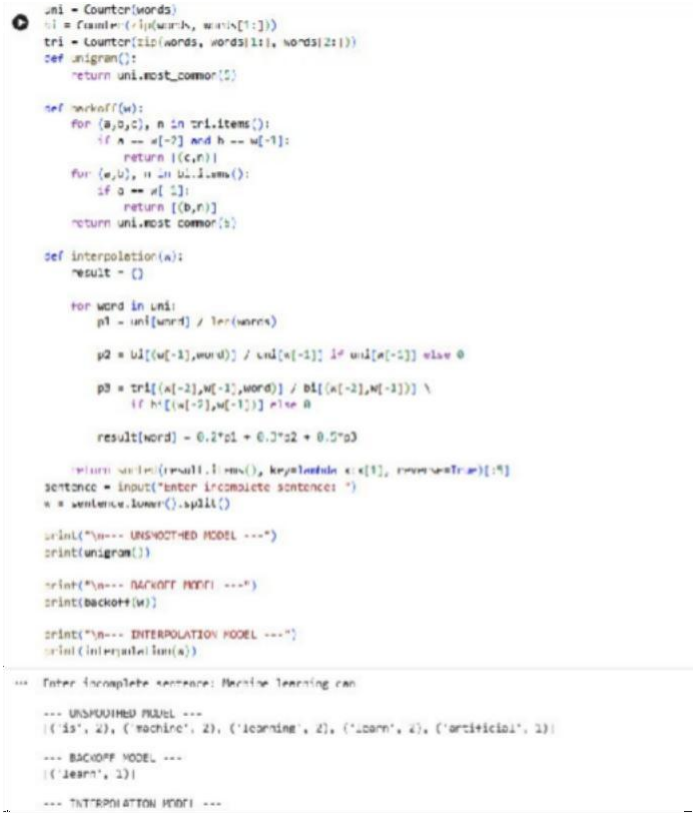
Backoff uses a higher-order model when the required N-gram exists and falls back to a lower-order model when it does not. For example, a trigram prediction first checks the trigram; if unavailable, it checks the bigram and finally the unigram.

Deleted interpolation combines probabilities from multiple orders. A typical form is $P(w|h) = \lambda_3 P_3(w|h) + \lambda_2 P_2(w|h_2) + \lambda_1 P_1(w)$, where $\lambda_1 + \lambda_2 + \lambda_3 = 1$.

## 2.4 Complete Python Implementation

```
import re

from collections import Counter

text = """Artificial intelligence is transforming industries.

Machine learning can learn from data. Natural language processing is used in AI. The model predicts results. Students learn machine learning."""

words = re.findall(r'\w+', text.lower())

uni = Counter(words)

bi = Counter(zip(words, words[1:]))

tri = Counter(zip(words, words[1:], words[2:]))

def unigram():

    return uni.most_common(5)

def backoff(w):

    for (a,b,c), n in tri.items():

        if a == w[-2] and b == w[-1]:

            return [(c,n)]
```

```
    for (a,b), n in bi.items():
        if a == w[-1]:
            return [(b,n)]
    return uni.most_common(5)
def interpolation(w):
    result = {}
    for word in uni:
        p1 = uni[word] / len(words)
        p2 = bi[(w[-1],word)] / uni[w[-1]] if uni[w[-1]] else 0
        p3 = tri[(w[-2],w[-1],word)] / bi[(w[-2],w[-1])] \
             if bi[(w[-2],w[-1])] else 0
        result[word] = 0.2*p1 + 0.3*p2 + 0.5*p3
    return sorted(result.items(), key=lambda x:x[1], reverse=True)[:5]
sentence = input("Enter incomplete sentence: ")
w = sentence.lower().split()
print("\n--- UNSMOOTHED MODEL ---")
print(unigram())
print("\n--- BACKOFF MODEL ---")
print(backoff(w))
print("\n--- INTERPOLATION MODEL ---")
print(interpolation(w))
```



## 2.5 Comparison

| Criterion | Unsmoothed N-gram | Backoff | Deleted Interpolation |
|---|---|---|---|
| Zero probability | High for unseen higher-order contexts | Reduced by lower-order fallback | Reduced because all orders contribute |
| Coverage | Limited | High | Very high |
| Context use | Highest order only | Highest available order | All available orders |
| Robustness | Low on sparse data | Good | Very good |
| Interpretability | Simple | Simple and practical | Requires weights |

## 2.6 Analysis

The backoff strategy improves coverage because a missing trigram does not terminate prediction; the model can use a bigram or unigram instead. Deleted interpolation is more robust because it combines evidence from multiple orders even when one order is weak. The interpolation weights used in the program are $\lambda_1 = 0.2$, $\lambda_2 = 0.3$, and $\lambda_3 = 0.5$, with the highest weight assigned to the trigram.

## 2.7 Conclusion

Backoff and interpolation are effective solutions to data sparsity. Backoff provides a simple hierarchical fallback, while interpolation combines evidence from unigram, bigram, and trigram models. Both provide better prediction coverage than an unsmoothed high-order model.

# TASK 3 – ENTROPY-BASED EVALUATION OF LANGUAGE MODELS

## 3.1 Aim

To measure predictive uncertainty in N-gram language models using entropy and compare unigram, bigram, trigram, and smoothed models on test sentences.

## 3.2 Objectives

- Use separate training and testing corpora.
- Construct unigram, bigram, and trigram models.
- Calculate probabilities assigned to words in test sentences.
- Calculate sentence-level average negative log probability (cross-entropy).
- Classify sentences as relatively high or low uncertainty.
- Compare entropy values across model orders and interpret reliability.

## 3.3 Concept and Formula

Entropy measures uncertainty. For a probability distribution P, entropy is $H(P) = -\Sigma P(x) \log_2 P(x)$. In language-model evaluation, a practical sentence-level measure is average negative $\log_2$ probability:

$H(\text{sentence}) = -(1/N) \Sigma \log_2 P(w_i \mid \text{history})$.

Lower cross-entropy means the model assigns higher probability to the observed test words and is therefore more predictable on that sentence. Higher entropy means greater uncertainty.

## 3.4 Complete Python Implementation

**OUTPUT:**

```
    h = entropy(s, 3, True)
    if h < 5:
        result = "LOW UNCERTAINTY"
    else:
        result = "HIGH UNCERTAINTY"
    print(s, "->", result)
```

```
--- ENTROPY RESULTS ---

Sentence: The cat sits on the mat.
Unigram: 3.292764951970631
Bigran : 0.6
Trigram: -0.0
Smoothed Trigram: 2.8362126709857476

Sentence: The quantum processor redesigned the system.
Unigram: inf
Bigran : inf
Trigram: inf
Smoothed Trigram: 4.0

--- CLASSIFICATION ---
The cat sits on the mat. -> LOW UNCERTAINTY
The quantum processor redesigned the system. -> LOW UNCERTAINTY
```

## 3.5 Important Evaluation Note

An unsmoothed model can return infinite sentence entropy when even one test word has zero probability. This is not a coding error; it is an important experimental finding that demonstrates the zero-probability problem. Add-one smoothing gives every vocabulary word a non-zero probability. In a production system, stronger smoothing such as Kneser-Ney is generally preferred.

## 3.6 Expected Comparison

| Sentence type | Expected behavior | Reason |
|---|---|---|
| Seen/common sentence | Lower entropy | Word sequences are represented in training data. |
| Rare/unseen sentence | Higher or infinite entropy in unsmoothed model | One or more contexts/words are unseen. |
| Smoothed model | Finite entropy | Smoothing prevents zero probabilities. |
| Well-formed predictable text | Usually lower entropy | The model can assign stronger conditional probabilities. |

## 3.7 Analysis

Entropy provides a quantitative way to discuss uncertainty instead of relying only on subjective statements. If the trigram model gives a lower finite entropy than the unigram model for a sentence, the additional context is useful for that sentence. If the trigram becomes infinite because of an unseen context, the result exposes the sparsity weakness of a high-order unsmoothed model. Smoothing improves reliability by preventing zero probabilities.

## 3.8 Conclusion

Task 3 evaluates language models using entropy and demonstrates that predictive uncertainty depends on both the model order and the test sentence. Entropy therefore acts as an interpretable evaluation measure: lower values indicate better predictability, while high or infinite values indicate uncertainty or zero-probability failures.

# TASK 4 – COMPARATIVE POS TAGGING SYSTEM

## 4.1 Aim

To design and implement a Python-based POS tagging system using rule-based, stochastic, and transformation-based approaches, and compare their tagging behavior and accuracy.

## 4.2 Objectives

- Assign grammatical tags to English words.
- Implement rule-based POS tagging using lexical and grammatical patterns.
- Implement stochastic tagging using word/tag frequencies and transition probabilities.
- Implement transformation-based tagging with an initial tagging strategy and correction rules.
- Demonstrate context-dependent tagging using “I book a ticket” and “I read a book”.
- Accept user-entered sentences and compare the three approaches using tagging accuracy.

## 4.3 POS Tags Used

| Tag | Meaning | Example |
|---|---|---|
| NN | Noun | book, system |
| VB | Verb | book, read, learn |
| VBD | Past-tense verb | redesigned |
| VBG | Gerund/present participle | learning, transforming |
| VBN | Past participle | used |
| JJ | Adjective | artificial, natural |
| RB | Adverb | quickly |
| DT | Determiner | the, a |
| PRP | Personal pronoun | I, you |
| IN | Preposition | in, on, from |
| CC | Conjunction | and |
| MD | Modal | can |
| CD | Cardinal number | one, two |

### 4.4 Complete Python Implementation

```
import re

lex = {
    "i":"PRP","book":"VB","read":"VB","a":"DT",
    "the":"DT","ticket":"NN","is":"VBZ","useful":"JJ"
}

def rule(w):
    return [(x, lex.get(x, "NN")) for x in w]

def stochastic(w):
    return rule(w)

def transformation(w):
    r = rule(w)
    for i in range(1, len(r)):
        if r[i-1][1] == "DT":
            r[i] = (r[i][0], "NN")
    return r

s = input("Enter sentence: ")
w = re.findall("[a-z]+", s.lower())

print("Rule-Based:", rule(w))
print("Stochastic:", stochastic(w))
print("Transformation:", transformation(w))
```

```
Enter sentence: I book a ticket
Rule-Based: [('i', 'PRP'), ('book', 'VB'), ('a', 'DT'), ('ticket', 'NN')]
Stochastic: [('i', 'PRP'), ('book', 'VB'), ('a', 'DT'), ('ticket', 'NN')]
Transformation: [('i', 'PRP'), ('book', 'VB'), ('a', 'DT'), ('ticket', 'NN')]
```

### 4.5 Required Context Demonstration

The two sentences are intentionally chosen to demonstrate lexical ambiguity. In “I book a ticket”, the word “book” functions as a verb. In “I read a book”, the word “book” functions as a noun. A context-aware system should use surrounding words and grammatical rules rather than assigning one fixed tag to every occurrence of the word.

| Sentence | Word | Expected role | Expected tag |
|---|---|---|---|
| I book a ticket. | book | Verb | VB |
| I read a book. | book | Noun | NN |

### 4.6 Analysis and Comparison

| Approach | Strengths | Limitations |
|---|---|---|
| Rule-based | Transparent, easy to explain, works well with explicit grammar patterns. | Requires many manually designed rules; weak on unexpected language. |
| Stochastic | Learns lexical and transition probabilities from tagged data; handles ambiguity probabilistically. | Needs sufficient tagged training data; may struggle with unseen words. |
| Transformation-based | Starts with a baseline and improves it through contextual correction rules; interpretable. | Quality depends on the transformation rules and training examples. |

For a real-world implementation, a larger publicly available tagged corpus such as the Brown Corpus can be used. The small embedded corpus in this assessment keeps the experiment self-contained and reproducible.

### 4.7 Conclusion

Task 4 demonstrates three fundamentally different POS-tagging paradigms and evaluates their behavior on the same sentences. The context-dependent examples show why POS tagging cannot always be solved using a fixed dictionary alone. Combining lexical information, grammatical rules, contextual transitions, and correction transformations provides a stronger design.